Project Interim Report

On

School Administration Management System (OOP Project)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

Krishna Bohare

ERP- RU-25-10699

Bachelor of Technology in Computer Science & Engineering with Cyber Security in association with EC council

Under the Guidance of

Soumik Karmakar

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

School Administration Management System

(Object-Oriented Programming Project Report)

---

1. Abstract

The School Administration Management System is a software-based application designed to manage and organize the core functionalities of an educational institution. This project is developed using Object-Oriented Programming (OOP) principles, focusing primarily on inheritance and polymorphism.

The system models real-world entities such as teachers and students through a hierarchical class structure. A base class Person is extended into specialized subclasses like Teacher and Student, each having distinct attributes and behaviors. Runtime polymorphism is demonstrated using dynamic method dispatch, allowing the system to behave differently based on object type.

This project enhances understanding of modular programming, code reusability, and scalability, forming a strong foundation for developing more complex management systems.

---

2. Introduction

In today's digital era, educational institutions require efficient systems to manage large volumes of data related to students, teachers, and administrative processes. Traditional manual systems are prone to errors, inefficiencies, and data mismanagement.

The School Administration Management System is designed to address these issues by providing a structured and object-oriented approach to data management. This project emphasizes the use of OOP concepts such as inheritance, polymorphism, and method overriding to simulate real-world relationships.

The system not only improves efficiency but also demonstrates how programming concepts can be applied to solve practical problems in education management.

---

3. Objectives

The key objectives of this project are:

- To design a structured system using OOP principles
- To implement inheritance for code reusability
- To demonstrate method overriding for specialized behavior
- To apply runtime polymorphism using dynamic method dispatch
- To create a scalable and maintainable class design
- To simulate real-world roles such as teachers and students

---

4. Scope of the Project

The scope of this project includes:

- Managing basic details of students and teachers

- Demonstrating role-based behavior using OOP
- Providing a simple console-based output system
- Serving as a foundational model for larger systems

Limitations:

- No database integration
- No graphical user interface
- Limited to demonstration of OOP concepts

---

5. System Requirements

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 or higher processor
- 500 MB free disk space

Software Requirements

- Java Development Kit (JDK)
- IDE (Eclipse / IntelliJ IDEA / NetBeans)
- Operating System (Windows/Linux/Mac)

---

6. Methodology

The development of this system follows a structured approach:

1. Requirement Analysis – Understanding system needs
2. System Design – Creating class structure
3. Implementation – Writing Java code
4. Testing – Verifying correctness
5. Documentation – Preparing the project report

The system is implemented using a bottom-up approach, starting from the base class and extending to derived classes.

---

7. Implementation

Base Class: Person

- Attributes: Name, Age
- Method: showRole()

Derived Class: Teacher

- Attributes: Subject, Salary
- Overrides showRole()

Derived Class: Student

- Attributes: Roll Number, Course

- Overrides showRole()

Polymorphism

- Implemented using an array of Person objects
- Allows dynamic behavior at runtime

---

8. OOP Concepts Used

1. Inheritance

Allows child classes to acquire properties from the parent class.

2. Encapsulation

Data is bundled within classes and accessed via methods.

3. Polymorphism

Enables one interface to be used for different data types.

4. Method Overriding

Allows subclasses to redefine parent class methods.

5. Dynamic Binding

Method calls are resolved at runtime.

---

9. Sentiment Classification (Conceptual Extension)

Although not directly implemented, sentiment classification can be integrated into the system for analyzing feedback from students and teachers.

Purpose:

- Analyze feedback (positive/negative)
- Improve teaching quality
- Monitor student satisfaction

Example:

- Input: “The teaching is excellent” → Positive
- Input: “The system is slow” → Negative

This feature can be added using machine learning models in future enhancements.

---

10. Project Structure

```
SchoolManagementSystem/
│
├── Person.java
├── Teacher.java
├── Student.java
├── Main.java
│
```

└── README.txt

---

11. Sample Input / Output

Input (Object Creation):

Teacher t = new Teacher("John", 40, "Math", 50000);

Student s = new Student("Alice", 20, 101, "CS");

Output:

I am a Teacher. I teach students.

I am a Student. I study subjects.

---

12. How to Run the Program

1. Install Java JDK
2. Open IDE (Eclipse/IntelliJ)
3. Create a new Java project
4. Add all class files
5. Compile the program
6. Run the Main class

Command Line Method:

javac Main.java

java Main

---

13. Gantt Chart (Project Timeline)

Task	Duration	Week
Requirement Analysis	2 Days	1
System Design	3 Days	1
Coding	5 Days	2
Testing	3 Days	2
Documentation	2 Days	3

---

14. Future Enhancements

- Add database integration (MySQL)
- Develop GUI using Java Swing/JavaFX
- Add Admin module
- Implement attendance tracking
- Add performance analytics

- Integrate sentiment analysis system

---

15. Conclusion

The School Administration Management System successfully demonstrates the practical implementation of Object-Oriented Programming concepts. Through inheritance and polymorphism, the system achieves modularity, flexibility, and code reusability.

This project serves as a strong academic foundation and can be expanded into a full-scale application with advanced features.

---

16. References

- Java Programming by Herbert Schildt
- Oracle Java Documentation
- Academic Notes and Lecture Materials
- Online tutorials and coding resources